

CRUD Operations in Entity Framework Database First Approach

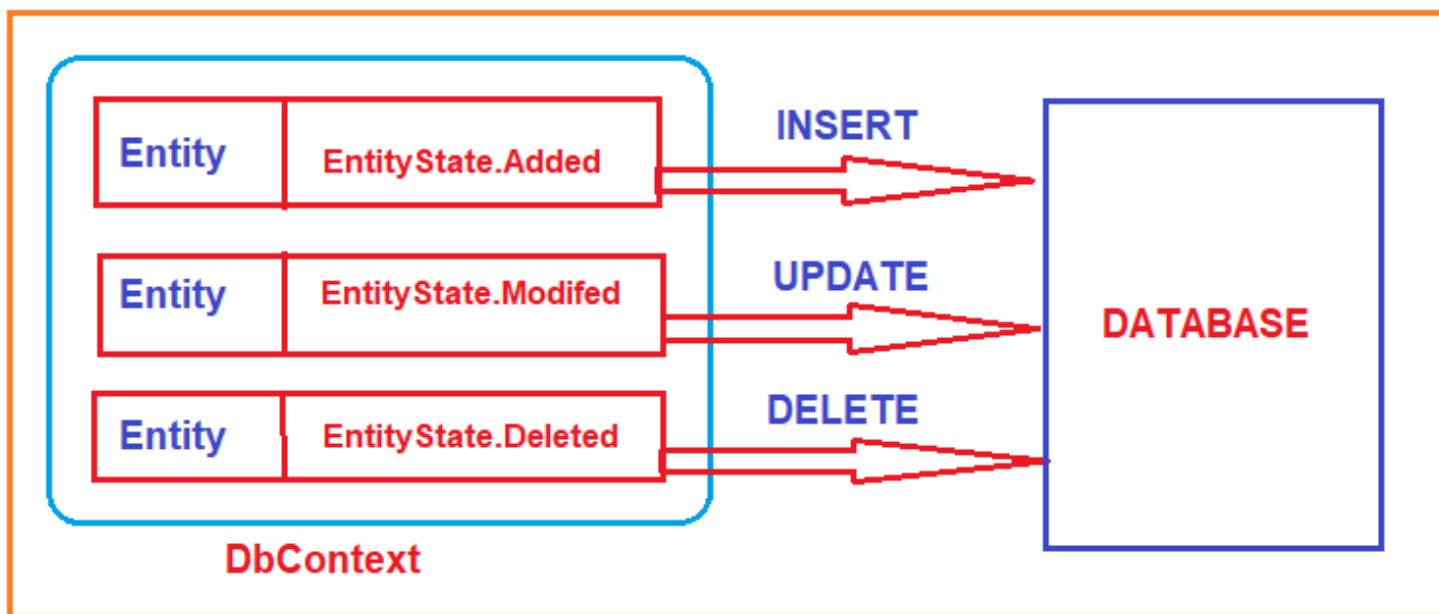
In this article, I am going to discuss **how to perform CRUD Operations in Entity Framework Database First Approach**. Please read our previous article where we discussed the [Relationships Between Entities in the Entity Framework](#). We are going to work with the same example that we created in our Introduction to Entity Framework Database First article. Please read our introduction to [Entity Framework Database First](#) article before proceeding to this article.

CRUD Operations in Entity Framework:

CRUD Operation means we need to perform Create, Update, Delete, and Retrieve Operations. In order to perform the Insert, Update, and Delete operations in Entity Framework we have two persistence scenarios i.e. connected and disconnected.

In the connected scenario, the same DbContext instance is used for retrieving and saving the entities, whereas this is different in the disconnected scenario. In this article, we will learn about saving data in the connected scenario, and in our upcoming articles, we will discuss how to perform the CRUD Operations in a disconnected scenario.

An entity that contains data in its scalar property will be either INSERTED, UPDATED, or DELETED, based on the State of the Entity. The Entity Framework builds and executes the INSERT, UPDATE, and DELETE SQL statements for the entities whose Entity State is Added, Modified, and Deleted respectively when the SaveChanges() method is called on the DbContext instance. The following diagram shows the CUD (Create, Update, Delete) operations in the connected scenario.



Note: In the connected scenario, an instance of DbContext class keeps track of all the entities and when an entity is created, modified, or deleted, it automatically sets the appropriate State for the entity.

Create Operation using Entity Framework:

Create Operation means we need to add a new object. Adding a new object using Entity Framework is very simple. First, you need to create an instance of the Entity which you want to add to the database. Once you created the instance of the entity then you need to call the Add method of DbSet class and pass the entity. Finally, call the SaveChanges method on the DbContext object which will insert the new record in the database. The following code lets

you add a new student to the database. The following example code is self-explained, so, please go through the comment lines.

```
using System;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create a new student which you want to add to the database
            var newStudent = new Student()
            {
                FirstName = "Pranaya",
                LastName = "Rout",
                StandardId = 1
            };
            //Create DbContext object
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Add Student Entity into Students DBset by calling the Add
                context.Students.Add(newStudent);
                //Now the Entity State will be in Added State
                Console.WriteLine($"Before SaveChanges Entity State:
{context.Entry(newStudent).State}");
                //If you want to see what SQL Statement it generates for
                Inserting the data,
                //please use the following statement, it will log the SQL
                Statement in the console window
                context.Database.Log = Console.Write;
                //Call SaveChanges method to save student into database
                context.SaveChanges();
                //Now the Entity State will change from Added State to
                Unchanged State
                Console.WriteLine($"After SaveChanges Entity State:
{context.Entry(newStudent).State}");
            }
            Console.ReadKey();
        }
    }
}
```

In the above example, **context.Students.Add(newStudent)** adds the newly created student instance to the context object with Added EntityState and when the **context.SaveChanges()** method is called, it builds and executes the INSERT statement to the database. Now, run the above code, and you will get the following output.

```

Before SaveChanges Entity State: Added
Opened connection at 08-12-2022 18:45:14 +05:30
Started transaction at 08-12-2022 18:45:14 +05:30
INSERT [dbo].[Student]([FirstName], [LastName], [StandardId])
VALUES (@0, @1, @2)
SELECT [StudentId]
FROM [dbo].[Student]
WHERE @@ROWCOUNT > 0 AND [StudentId] = scope_identity()
-- @0: 'Pranaya' (Type = AnsiString, Size = 100)
-- @1: 'Rout' (Type = AnsiString, Size = 100)
-- @2: '1' (Type = Int32)
-- Executing at 08-12-2022 18:45:14 +05:30
-- Completed in 19 ms with result: SqlDataReader

Committed transaction at 08-12-2022 18:45:14 +05:30
Closed connection at 08-12-2022 18:45:14 +05:30
After SaveChanges Entity State: Unchanged

```

← Captured by the following statement
context.Database.Log = Console.Write;

context.Database.Log = Console.Write; This statement will log when the connection is established with the database, it will log when the transaction is started, it will log the generated SQL Statement, then it will log when the transaction is committed or rolled back and finally it will log when the connection is closed. Here, it will log everything in the Console window as we are passing the Console Class Write method.

Update Operation in Entity Framework:

As we already discussed in the connected environment, the Entity Framework keeps track of all the entities retrieved using the context object. Therefore, when we edit any entity data, the Entity Framework will automatically mark the Entity State as Modified and when the SaveChanges method is called, it updates the updated data into the underlying database. The following code is used to change the first name and last name of the student whose id is 3. The following example code is self-explained, so, please go through the comment lines.

```

using System;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create DbContext object
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                // Retrieve the student whose id is 3
                var student = context.Students.Find(3);
                //At this point Entity State will be Unchanged
                Console.WriteLine($"Before Updating Entity State:
{context.Entry(student).State}");
                //Update the first name and last name
                student.FirstName = "Sanju";
                student.LastName = "Samson";
                //At this point Entity State will be Modified
                Console.WriteLine($"After Updating Entity State:
{context.Entry(student).State}");
                //If you want to see what SQL Statement it generates for
                Updating the data,

```

```

        //please use the following statement, it will log the SQL
        Statement in the console window
        context.Database.Log = Console.Write;
        //Call SaveChanges method to update student data into
        database
        context.SaveChanges();
        //Now the Entity State will change from Modified State to
        Unchanged State
        Console.WriteLine($"After SaveChanges Entity State:
        {context.Entry(student).State}");
    }
    Console.ReadKey();
}
}
}
}

```

In the above example, first, we retrieve the student from the database using the DbSet Find method which generates a SELECT SQL query. As soon as we modify the FirstName and LastName, the context object sets its Entity State to Modified and when we call the SaveChanges() method, it builds and executes the Update SQL statement in the database. Now, run the above code, and you will get the following output.

```

Before Updating Entity State: Unchanged
After Updating Entity State: Modified
Opened connection at 08-12-2022 18:58:40 +05:30
Started transaction at 08-12-2022 18:58:40 +05:30
UPDATE [dbo].[Student]
SET [FirstName] = @0, [LastName] = @1
WHERE ([StudentId] = @2)
-- @0: 'Sanju' (Type = AnsiString, Size = 100)
-- @1: 'Samson' (Type = AnsiString, Size = 100)
-- @2: '3' (Type = Int32)
-- Executing at 08-12-2022 18:58:40 +05:30
-- Completed in 3 ms with result: 1

Committed transaction at 08-12-2022 18:58:40 +05:30
Closed connection at 08-12-2022 18:58:40 +05:30
After SaveChanges Entity State: Unchanged

```

← Only Updating the FirstName and LastName Columns based on the Primary key column

Note: The context object in Entity Framework keeps track of the columns of an entity that are modified and based on the modified columns, it will generate the UPDATE SQL statement, and here in the where condition it is going to use the primary key column. If one column value is not updated, then it will not include that column while updating in the database.

Delete Operation in Entity Framework:

To delete an entity using Entity Framework, we need to use the Remove method of the DbSet object. The DbSet Remove method works for both existing and newly added entities which are tracked by the context object. When we call the Remove method by passing an entity that has been added in the context object but not yet saved to the database will cancel the addition of the entity i.e. it will mark the entity state as Detached which means it is no longer tracked by the context object. The following code shows we added one entity in the context object but we did not call the SaveChanges method and then we call the Remove method. In this case, the entity is removed from the change tracker and is no longer tracked by the DbContext object as the Remove method marks the state of the entity as Detached.

```
using System;
```

```

namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            var newStudent = new Student()
            {
                FirstName = "Test1",
                LastName = "Test2",
                StandardId = 1
            };
            //Create DbContext object
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //If you want to Capture SQL Statements generates by the
                context object use the following statement
                //it will log the SQL Statements in the console window
                context.Database.Log = Console.Write;
                //Add the entity to the context object
                context.Students.Add(newStudent);
                //At this point, the Entity State is Addedd
                Console.WriteLine($"Entity State Before Removing:
{context.Entry(newStudent).State}");
                //The following statement will cancel the addition of the
                Entity
                context.Students.Remove(newStudent);
                //At this point, the Entity State will be in Detached state
                Console.WriteLine($"Entity State After Removing:
{context.Entry(newStudent).State}");

                //SaveChanges method will not do anything in the database
                context.SaveChanges();
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```

Entity State Before Removing: Added
Entity State After Removing: Detached

```

Calling the Remove method on an existing entity that is being tracked by the context object will not immediately delete the entity, rather it will mark the entity state as Deleted, and when we call the SaveChanged method, then the Entity is going to be deleted from the database and once the Save Changes method executed successfully, it will mark the entity state as Detached which means now the entity is not going to be tracked by the context object. In the following example, we are removing a student from the database whose StudentId is 6. The following example code is self-explained, so, please go through the comment lines.

```

using System;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create DbContext object
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                var student = context.Students.Find(6);
                //At this point the Entity State will be Unchanged
                Console.WriteLine($"Entity State Before Removing:
{context.Entry(student).State}");
                //The following statement mark the Entity State as Deleted
                context.Students.Remove(student);
                //At this point, the Entity State will be in Deleted state
                Console.WriteLine($"Entity State After Removing:
{context.Entry(student).State}");
                //If you want to Capture SQL Statements generates by the
                context object use the following statement
                //it will log the SQL Statements in the console window
                context.Database.Log = Console.Write;
                //SaveChanges method will delete the Entity from the database
                context.SaveChanges();
                //Once the SaveChanges Method executed successfully,
                //the Entity State will be in Detached state
                Console.WriteLine($"Entity State After Removing:
{context.Entry(student).State}");
            }
            Console.ReadKey();
        }
    }
}

```

In the above example, **context.Students.Remove(student)** marks the **student** entity object as Deleted and when we call the **SaveChanges** method on the Context object, the Entity Framework will build and execute the DELETE statement in the database, and then it will mark the entity state as Detached means it is no longer being tracked by the context object. Now, run the above code, and you should get the following output.

```

Entity State Before Removing: Unchanged
Entity State After Removing: Deleted
Opened connection at 08-12-2022 19:28:47 +05:30
Started transaction at 08-12-2022 19:28:47 +05:30
DELETE [dbo].[Student]
WHERE ([StudentId] = @0)
-- @0: '16' (Type = Int32)
-- Executing at 08-12-2022 19:28:47 +05:30
-- Completed in 4 ms with result: 1

```

```

Committed transaction at 08-12-2022 19:28:47 +05:30
Closed connection at 08-12-2022 19:28:47 +05:30
Entity State After Removing: Detached

```



**Generating the DELETE SQL Statement
based on the Primary Key Column**

Read Operation in Entity Framework:

Retrieving data from the database table is very simple. In the below example, we are retrieving all the records from the Student database table and then displaying the FirstName and LastName in the Console window.

```
using System;
using System.Linq;
namespace DBFirstApproach
{
    class Program
    {
        static void Main(string[] args)
        {
            //Create DbContext object
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //If you want to Capture SQL Statements generates by the
                context object, use the following statement
                //it will log the SQL Statements in the console window
                context.Database.Log = Console.Write;
                //Fetching All Records from the Student Database Table
                var AllStudents = context.Students.ToList();
                Console.WriteLine();
                //Printing the First Name and Last Name of All Students
                foreach (var student in AllStudents)
                {
                    Console.WriteLine($"FirstName : {student.FirstName}
LastName : {student.LastName}");
                }
            }
            Console.ReadKey();
        }
    }
}
```

In the above example, context.Students.ToList() retrieves all the records from the student database table and then we are using a for each loop to iterate through the student collection and print the First Name and Last Name of the student. Now, run the above code, and you should get the following output. Here, you can also see that it is generating a SELECT SQL Statement.

```
Opened connection at 08-12-2022 19:38:18 +05:30
SELECT
    [Extent1].[StudentId] AS [StudentId],
    [Extent1].[FirstName] AS [FirstName],
    [Extent1].[LastName] AS [LastName],
    [Extent1].[StandardId] AS [StandardId]
FROM [dbo].[Student] AS [Extent1]
-- Executing at 08-12-2022 19:38:18 +05:30
-- Completed in 8 ms with result: SqlDataReader

Closed connection at 08-12-2022 19:38:18 +05:30
```

```
FirstName : Virat LastName : Kohli
FirstName : Rohit LastName : Sharma
FirstName : Sanju LastName : Samson
FirstName : Smriti LastName : Mandana
```

In the next article, I am going to discuss [Querying in Entity Framework](#) using different approaches. In this article, I try to explain How to Perform **CRUD Operations using Entity Framework Database First Approach** and I hope you enjoyed this How to Perform CRUD Operations using the Entity Framework article. Please give your valuable feedback and suggestions about this article.